

Guia especializada para Examen 2

Patrones de Software y Programacion

Formulario de estudio: ArchUnit, JUnit, Mockito, Vert.x y arquitectura

Junio 2026

Objetivo de esta guia. Prepararte para una evaluacion practica similar a la formativa: leer un proyecto Java degradado, identificar violaciones arquitecturales, refactorizar separando responsabilidades y demostrar la correccion con ArchUnit, JUnit, Mockito y Vert.x. No es una guia de memoria: es un formulario de decision y escritura de codigo.

Índice

1. Mapa mental de la prueba	4
1.1. Hipotesis de evaluacion	4
1.2. Patron repetido en los enunciados	4
1.3. Regla de oro para resolver	4
2. Analisis de evaluaciones revisadas	4
2.1. Formativo II PSP 2026: solicitudes academicas	5
2.2. Evaluacion II 2025: monitoreo de calidad del aire	6
2.3. Evaluacion II 2024: polizas, XML y modularidad	7
2.4. Formativa II 2025	8
3. Arquitectura: formulario de diagnostico	8
3.1. Conceptos base	8
3.2. Dos y tres capas	8
3.3. MVC en examen	9
3.4. Microservicios: cuando si y cuando no	9
3.5. Pipes and Filters	9
4. ArchUnit: formulario DSL	9
4.1. Idea central	9
4.2. Patron S-R-C-E-X	10
4.3. Plantilla minima	10
4.4. Patrones de paquetes	10
4.5. Reglas frecuentes para la prueba	10
4.5.1. Controller no depende de infra	10
4.5.2. View no depende de JPA ni infra	11
4.5.3. Application no depende de JavaFX/Javalin	11
4.5.4. Domain solo depende de domain y Java	11
4.5.5. Web solo accede a service y model	11

4.5.6. Service no depende de web	11
4.5.7. Repository no depende de web ni service	11
4.6. Layered Architecture	12
4.7. Onion/Hexagonal Architecture	12
4.8. Slices: ciclos por feature	12
4.9. Naming rules	12
4.10 Checklist antes de entregar una regla ArchUnit	13
5. JUnit: formulario de tests unitarios	13
5.1. Idea central	13
5.2. JUnit 4: plantilla	13
5.3. JUnit 5: plantilla	13
5.4. Estructura AAA	14
5.5. Nombres de tests recomendados	14
5.6. Casos minimos para dominio de solicitudes	14
5.7. Casos minimos para sensores	14
5.8. Aserciones frecuentes	15
6. Mockito: formulario DSL	15
6.1. Idea central	15
6.2. JUnit 4 + Mockito	15
6.3. JUnit 5 + Mockito	16
6.4. Verificar que NO se invoca dependencia externa	16
6.5. Mockito para EntityManager y TypedQuery	16
6.6. ArgumentCaptor	17
6.7. Errores frecuentes con Mockito	18
7. Vert.x: formulario DSL	18
7.1. Conceptos base	18
7.2. Arranque comun	18
7.3. Point-to-Point	18
7.4. Publish/Subscribe	19
7.5. Request/Response	20
7.6. Regla arquitectural para Vert.x	20
8. Plantillas de solucion por tipo de pregunta	21
8.1. Pregunta tipo A: "El test ArchUnit falla"	21
8.2. Pregunta tipo B: "Crear test unitario"	21
8.3. Pregunta tipo C: "Crear test con Mockito"	21
8.4. Pregunta tipo D: "No debe invocarse servicio externo si falla"	21
8.5. Pregunta tipo E: "Migrar a Vert.x pub/sub"	22
9. Ejercicios tipo prueba con solucion orientativa	22
9.1. Ejercicio 1: Controller acoplado a JPA	22
9.2. Ejercicio 2: Mockito y evento de cambio de estado	23
9.3. Ejercicio 3: Vert.x pub/sub con sensor	23
9.4. Ejercicio 4: Poliza acoplada a XML	24
10 Checklist final de resolucion en 90 minutos	25
11 Banco rapido de respuestas teoricas	26
12 Resumen ultra-condensado	26

13 Fuentes revisadas

27

1 Mapa mental de la prueba

1.1 Hipotesis de evaluacion

A partir de la prueba formativa 2026 y evaluaciones anteriores, la prueba real probablemente no pedira escribir APIs aisladas sin contexto. Lo mas probable es que entregue un proyecto base con deuda tecnica y pida una combinacion de estas acciones:

1. **Diagnosticar arquitectura:** detectar dependencias indebidas entre paquetes o capas.
2. **Refactorizar:** mover logica desde controladores o verticles hacia servicios de aplicacion/dominio.
3. **Escribir reglas ArchUnit:** convertir la arquitectura esperada en reglas ejecutables.
4. **Crear tests JUnit:** probar reglas de dominio y casos de uso sin infraestructura real.
5. **Usar Mockito:** simular repositorios, servicios externos, EntityManager, TypedQuery o publicadores.
6. **Implementar Vert.x:** usar EventBus con point-to-point, publish/subscribe o request/response.
7. **Justificar:** explicar por que la solucion reduce acoplamiento y respeta responsabilidades.

1.2 Patron repetido en los enunciados

Elemento	Que suele pedir
Arquitectura	Web/controller no debe saltarse service/application; domain no debe depender de infra; repository no debe conocer web.
Testing unitario	Probar reglas de negocio, estados validos/invalidos, calculos y excepciones.
Mockito	Aislar BD, JPA, servicios externos, publicadores de eventos o notificaciones.
Vert.x	Publicar eventos o mensajes JSON; consumer debe delegar a servicio, no guardar directamente.
ArchUnit	Reglas por paquetes, layered architecture, no cycles, onion/hexagonal, naming.
Refactor	Introducir service/application/ports para desacoplar control, dominio e infraestructura.

1.3 Regla de oro para resolver

Cuando veas una clase que mezcla responsabilidades, aplica esta pregunta:

Si cambio la tecnologia externa, que clases no deberian cambiar?

Si al cambiar JPA, JavaFX, Javalin, XML, Vert.x o un servicio externo debes modificar dominio o casos de uso, hay acoplamiento indebido. La solucion natural es introducir una abstraccion: servicio, puerto, interfaz o adaptador.

2 Analisis de evaluaciones revisadas

2.1 Formativo II PSP 2026: solicitudes academicas

Contexto

Sistema Java para solicitudes academicas con interfaz JavaFX. El flujo visible parece MVC:

```
View -> Controller -> Model
```

Pero internamente el controlador conoce infraestructura:

```
Controller -> infra.jpa  
Controller -> infra.external
```

La arquitectura esperada conserva el comportamiento MVC, pero con separacion interna:

```
View -> Controller -> Application/Service -> Domain Model -> Repository/Infrastructure
```

Lo que realmente evalua

1. Identificar que SolicitudController tiene dependencias tecnicas indebidas.
2. Mover coordinacion de caso de uso a application o service.
3. Mantener entidades y reglas en domain.
4. Mantener JPA, Vert.x y servicios externos fuera del controlador.
5. Escribir al menos una regla ArchUnit adicional coherente.
6. Crear pruebas JUnit para reglas de Estudiante, Solicitud y cambio de estado.
7. Usar Mockito para verificar guardado, publicacion de evento y notificacion externa.
8. Usar Vert.x preferentemente con publish/subscribe para evento de cambio de estado.

Solucion arquitectural tipo

```
cl.ucn.solicitudes  
  view  
    SolicitudView.java  
  controller  
    SolicitudController.java  
  application  
    SolicitudService.java  
  port  
    SolicitudRepository.java  
    SolicitudEventPublisher.java  
    NotificadorSolicitud.java  
  domain  
    Estudiante.java  
    Solicitud.java  
    EstadoSolicitud.java  
  infra  
    jpa  
      JpaSolicitudRepository.java  
    vertx  
      VertxSolicitudEventPublisher.java  
      HistorialSolicitudVerticle.java  
  external  
    EmailNotificadorSolicitud.java
```

Dependencia correcta:

```

Controller -> SolicitudService
SolicitudService -> ports + domain
infra.jpa -> implements SolicitudRepository
infra.vertx -> implements SolicitudEventPublisher
infra.external -> implements NotificadorSolicitud

```

Dependencia incorrecta:

```

Controller -> JpaSolicitudRepository
Controller -> EmailNotificadorSolicitud
Domain -> Vertx
Domain -> JPA

```

2.2 Evaluacion II 2025: monitoreo de calidad del aire

Contexto

Proyecto con Javalin, Hibernate y SQLite. Estructura relevante:

```

cl.ucn.airmonitor
  configuration
  model
  repository
  service
  web
  vertx

```

Lo que evalua

1. Reglas ArchUnit para controlar dependencias entre web, service, repository y model.
2. Tests unitarios para SensorServiceImplTest: registro de mediciones y calculo de promedios.
3. Mockito con EntityManager y TypedQuery<Measurement> para no tocar SQLite.
4. Vert.x pub/sub: SensorPublisherVerticle publica JSON; MeasurementConsumerVerticle consume, valida y delega en servicio.

Error tipico que la prueba castiga

```

1 public class MeasurementConsumerVerticle extends AbstractVerticle {
2     private EntityManager em; // MAL: el verticle no debe persistir directo
3
4     @Override
5     public void start() {
6         vertx.eventBus().consumer("air.monitor.measurements", msg -> {
7             Measurement m = parse(msg.body());
8             em.persist(m); // MAL: logica/persistencia en adaptador de mensajeria
9         });
10    }
11 }

```

Solucion tipo

```

1 public class MeasurementConsumerVerticle extends AbstractVerticle {
2     private final MeasurementService measurementService;
3
4     public MeasurementConsumerVerticle(MeasurementService measurementService) {
5         this.measurementService = measurementService;
6     }
7 }

```

```

6      }
7
8      @Override
9      public void start() {
10         vertx.eventBus().<JsonObject>consumer("air.monitor.measurements", msg -> {
11             JsonObject json = msg.body();
12             if (!isValid(json)) {
13                 return;
14             }
15             measurementService.registrarMedicion(
16                 json.getInteger("sensorId"),
17                 json.getString("buildingCode"),
18                 json.getString("timestamp"),
19                 json.getDouble("temperature"),
20                 json.getDouble("humidity"),
21                 json.getDouble("mp25")
22             );
23         });
24     }
25
26     private boolean isValid(JsonObject json) {
27         return json.containsKey("sensorId")
28             && json.containsKey("temperature")
29             && json.containsKey("humidity")
30             && json.containsKey("mp25")
31             && json.containsKey("timestamp");
32     }
33 }

```

2.3 Evaluacion II 2024: polizas, XML y modularidad

Contexto

El problema central es que Poliza queda acoplada al XML. Eso impide reutilizar validacion y persistencia en otra aplicacion, porque el formato de entrada esta mezclado con el dominio.

Solucion conceptual

Separar el dominio del mecanismo de construccion:

```

org.poliza
  Poliza.java
  ConstructorPoliza.java
  ManejadorDefectoPoliza.java    (si pertenece al parsing, puede moverse/revisarse)

org.polizaxml
  ConstructorPolizaXML.java

```

```

1 public interface ConstructorPoliza {
2     Poliza construir();
3 }

```

```

1 public class Poliza {
2     public static Poliza construirPoliza(ConstructorPoliza constructor) {
3         return constructor.construir();
4     }
5 }

```

Idea evaluada: Poliza no debe importar javax.xml, org.xml.sax ni detalles de parsing. El XML se vuelve un adaptador de entrada.

2.4 Formativa II 2025

Esta formativa refuerza el mismo patron: proyecto con Javalin, Hibernate y SQLite; pruebas con Mockito, reglas ArchUnit, JUnit con assertEquals y migracion parcial a Vert.x. El mensaje de fondo es que la prueba combina **arquitectura verificable** + **testabilidad** + **mensajeria reactiva**.

3 Arquitectura: formulario de diagnostico

3.1 Conceptos base

Concepto	Definicion operativa para la prueba
Arquitectura	Decisiones de alto nivel sobre paquetes, modulos, servicios, capas y dependencias.
Capa	Grupo de responsabilidades; una capa superior usa servicios de la inferior, no al revés.
MVC tradicional	Vista + Controlador forman interfaz grafica; Modelo contiene datos y logica. Pensado originalmente para aplicaciones desktop monoliticas.
MVC Web	Suele parecerse a tres capas: vista web, aplicacion/control, datos.
SPA	Cliente web mas autonomo; se parece mas al MVC tradicional en el browser.
Microservicio	Modulo o conjunto de modulos que corre como proceso independiente en runtime.
Mensajeria	Los emisores no conocen directamente a los receptores; se comunican por cola, broker o event bus.

3.2 Dos y tres capas

Tres capas classicas

Presentacion -> Aplicacion/Negocio -> Datos

Ventaja: separa responsabilidades; facilita reemplazo de UI o persistencia. **Riesgo:** si presentacion accede directo a datos, se rompe la arquitectura.

Dos capas

Cliente: interfaz + logica
Servidor BD: datos

Riesgo: mucha logica queda en cliente; menor control y peor mantenibilidad.

3.3 MVC en examen

Componente	Debe hacer	No debe hacer
View	Mostrar datos, capturar entrada, delegar accion.	Abrir BD, usar JPA, aplicar reglas de negocio complejas.
Controller	Coordinar vista y caso de uso.	Instanciar repositorios concretos, llamar servicios externos tecnicos, guardar directo.
Application/Service	Ejecutar caso de uso, coordinar dominio y puertos.	Conocer JavaFX/Javalin si puede evitarse.
Domain	Reglas de negocio, entidades, invariantes.	Depender de JPA, Vert.x, JavaFX, Javalin, XML.
Infrastructure	Implementar persistencia, mensajeria, servicios externos.	Contaminar el dominio con detalles tecnicos.

3.4 Microservicios: cuando si y cuando no

Decision	Criterio
Usar monolito primero	Proyecto inicial, poca carga, equipo pequeño, dominio todavia inestable.
Migrar gradualmente	El monolito atrasa releases o solo un modulo necesita escalar.
No usar microservicios	Si no puedes asumir latencia, complejidad distribuida, observabilidad, transacciones distribuidas.
Dato por microservicio	Recomendable: cada microservicio posee su propia persistencia para evitar acoplamiento de datos.
BD compartida	Mala señal: acopla servicios y dificulta evolucion independiente.

3.5 Pipes and Filters

Aunque no aparece como foco principal en las evaluaciones revisadas, puede aparecer como pregunta conceptual o refactor.

```
Input -> Filter 1 -> Filter 2 -> Filter 3 -> Output
```

Uso tipico: procesamiento secuencial de datos, validaciones encadenadas, transformaciones, OCR, compiladores, ETL.

Criterio: cada filtro debe tener una responsabilidad clara y producir una salida para el siguiente filtro. Si un filtro conoce todos los pasos del pipeline, se pierde el patron.

4 ArchUnit: formulario DSL

4.1 Idea central

ArchUnit permite escribir tests que comparan la arquitectura planeada con la arquitectura real del codigo compilado. Para la prueba, piensa asi:

```
Decision arquitectural -> Regla ArchUnit -> mvn test falla si se rompe
```

4.2 Patron S-R-C-E-X

Letra	Uso
S: Scope	@AnalyzeClasses(packages = "cl.ucn...") define que se analiza.
R: Referente	classes().that()... selecciona las clases sujeto.
C: Condicion	should()... expresa permitido/prohibido.
E: Evidencia	because("...") justifica la decision.
X: Excepciones	freeze(...) o excepciones acotadas para deuda tecnica.

4.3 Plantilla minima

```

1 import com.tngtech.archunit.junit.AnalyzeClasses;
2 import com.tngtech.archunit.junit.ArchTest;
3 import com.tngtech.archunit.lang.ArchRule;
4
5 import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.*;
6
7 @AnalyzeClasses(packages = "cl.ucn.sistema")
8 class ArchitectureTest {
9
10     @ArchTest
11     static final ArchRule dominio_no_depends_de_infra =
12         noClasses().that().resideInAPackage("..domain..")
13             .should().dependOnClassesThat().resideInAnyPackage(
14                 "..infra..", "..web..", "..controller..")
15             .because("el dominio debe ser independiente de detalles tecnicos");
16 }

```

4.4 Patrones de paquetes

Patron	Significado
"..domain.."	Cualquier paquete que contenga domain.
cl.ucn.bank.."	Todo bajo cl.ucn.bank.
"..adapter.in.web.."	Adaptadores de entrada web.
"..adapter.out.persistence.."	Adaptadores de salida a persistencia.
"java.."	Biblioteca estandar Java.
"javax..", "jakarta.."	APIs externas. Revisar si se permiten en model.

4.5 Reglas frecuentes para la prueba

4.5.1 Controller no depende de infra

```

1 @ArchTest
2 static final ArchRule controller_no_depends_de_infra =
3     noClasses().that().resideInAPackage("..controller..")
4         .should().dependOnClassesThat().resideInAPackage("..infra..")
5         .because("el controlador debe coordinar casos de uso, no detalles tecnicos");

```

4.5.2 View no depende de JPA ni infra

```
1 @ArchTest
2 static final ArchRule view_no_depends_de_persistencia =
3     noClasses().that().resideInAPackage("..view..")
4         .should().dependOnClassesThat().resideInAnyPackage(
5             "..infra.jpa..", "jakarta.persistence..", "javax.persistence..")
6         .because("la vista no debe conocer persistencia");
```

4.5.3 Application no depende de JavaFX/Javalin

```
1 @ArchTest
2 static final ArchRule application_no_depends_de_ui =
3     noClasses().that().resideInAPackage("..application..")
4         .should().dependOnClassesThat().resideInAnyPackage(
5             "javafx..", "io.javalin..")
6         .because("los casos de uso no deben depender del framework de entrada");
```

4.5.4 Domain solo depende de domain y Java

```
1 @ArchTest
2 static final ArchRule dominio_puro =
3     classes().that().resideInAPackage("..domain..")
4         .should().onlyDependOnClassesThat().resideInAnyPackage(
5             "..domain..", "java..")
6         .because("el dominio concentra reglas y debe ser testeable sin infraestructura");
```

4.5.5 Web solo accede a service y model

```
1 @ArchTest
2 static final ArchRule web_solo_depends_de_service_y_model =
3     classes().that().resideInAPackage("..web..")
4         .should().onlyDependOnClassesThat().resideInAnyPackage(
5             "..web..", "..service..", "..model..", "java..",
6             "io.javalin..", "org.slf4j..")
7         .because("web debe delegar en service y no saltarse capas");
```

4.5.6 Service no depende de web

```
1 @ArchTest
2 static final ArchRule service_no_depends_de_web =
3     noClasses().that().resideInAPackage("..service..")
4         .should().dependOnClassesThat().resideInAPackage("..web..")
5         .because("la capa de negocio no debe conocer la capa de entrada");
```

4.5.7 Repository no depende de web ni service

```
1 @ArchTest
2 static final ArchRule repository_no_depends_de_capas_superiores =
3     noClasses().that().resideInAPackage("..repository..")
4         .should().dependOnClassesThat().resideInAnyPackage("..web..", "..service..")
5         .because("repository debe encapsular datos, no llamar casos de uso ni controladores");
```

4.6 Layered Architecture

Usa esto cuando el enunciado define capas classicas.

```

1 import static com.tngtech.archunit.library.Architectures.layeredArchitecture;
2
3 @ArchTest
4 static final ArchRule arquitectura_en_capas =
5     layeredArchitecture()
6         .consideringAllDependencies()
7         .layer("Web").definedBy("..web..")
8         .layer("Service").definedBy("..service..")
9         .layer("Repository").definedBy("..repository..")
10        .layer("Model").definedBy("..model..")
11        .whereLayer("Web").mayNotBeAccessedByAnyLayer()
12        .whereLayer("Service").mayOnlyBeAccessedByLayers("Web")
13        .whereLayer("Repository").mayOnlyBeAccessedByLayers("Service")
14        .because("el flujo debe ser web -> service -> repository -> model");

```

Cuidado: layeredArchitecture() puede necesitar ajustes si model es usado por todas las capas. No fuerces una regla imposible; expresa la arquitectura real esperada.

4.7 Onion/Hexagonal Architecture

Usa esto cuando hay domain, application, ports, adapter.in, adapter.out.

```

1 import static com.tngtech.archunit.library.Architectures.onionArchitecture;
2
3 @ArchTest
4 static final ArchRule arquitectura_hexagonal =
5     onionArchitecture()
6         .domainModels("..domain.model..")
7         .domainServices("..domain.service..")
8         .applicationServices("..application..")
9         .adapter("web", "..adapter.in.web..")
10        .adapter("persistence", "..adapter.out.persistence..")
11        .adapter("messaging", "..adapter.out.messaging..")
12        .because("las dependencias deben apuntar hacia el nucleo del negocio");

```

4.8 Slices: ciclos por feature

```

1 import static com.tngtech.archunit.library.dependencies.SlicesRuleDefinition.slices;
2
3 @ArchTest
4 static final ArchRule sin_ciclos_por_modulo =
5     slices().matching("cl.ucn.sistema.*")
6         .should().beFreeOfCycles()
7         .because("los ciclos impiden modularidad y evolucion controlada");

```

4.9 Naming rules

```

1 @ArchTest
2 static final ArchRule controladores_terminan_en_controller =
3     classes().that().resideInAPackage("..controller..")
4         .should().haveSimpleNameEndingWith("Controller")
5         .because("el nombre comunica el rol arquitectural");

```

```

1 @ArchTest
2 static final ArchRule servicios_terminan_en_service =
3     classes().that().resideInAPackage("..service..")
4         .should().haveSimpleNameEndingWith("Service")
5         .because("las clases de service deben ser reconocibles");

```

4.10 Checklist antes de entregar una regla ArchUnit

1. La regla expresa una decision real, no una preferencia estetica arbitraria.
2. El paquete base de @AnalyzeClasses coincide con el proyecto.
3. Los patrones ". . x . ." no son demasiado amplios ni demasiado estrechos.
4. La regla permite dependencias legitimas como Java, JUnit, JPA en entidades si el proyecto lo exige.
5. Incluye because(. . .) claro.
6. Si falla, el mensaje ayuda a encontrar la clase culpable.

5 JUnit: formulario de tests unitarios

5.1 Idea central

Un test unitario prueba una unidad pequeña y debe ser rapido, independiente, deterministico, auto-verificable y oportuno. En la prueba, evita tests que dependan de BD real, red, archivos externos o temporizadores reales salvo que el enunciado lo permita.

5.2 JUnit 4: plantilla

La formativa 2026 menciona JUnit 4 para tests unitarios. Usa esta plantilla si el pom.xml trae JUnit 4.

```
1 import org.junit.Before;
2 import org.junit.Test;
3
4 import static org.junit.Assert.*;
5
6 public class SolicitudTest {
7     private Estudiante estudiante;
8
9     @Before
10    public void setUp() {
11        estudiante = new Estudiante("Ana", "ana@alumnos.ucn.cl");
12    }
13
14    @Test
15    public void crearSolicitudValida_dejaEstadoInicialPendiente() {
16        Solicitud solicitud = new Solicitud(estudiante, "RETIRO", "Solicitud de retiro");
17
18        assertEquals(EstadoSolicitud.PENDIENTE, solicitud.getEstado());
19    }
20
21    @Test(expected = IllegalArgumentException.class)
22    public void crearSolicitudConEstudianteNulo_lanzaExcepcion() {
23        new Solicitud(null, "RETIRO", "Solicitud de retiro");
24    }
25 }
```

5.3 JUnit 5: plantilla

Usa esta si el proyecto trae Jupiter.

```
1 import org.junit.jupiter.api.BeforeEach;
2 import org.junit.jupiter.api.Test;
3
4 import static org.junit.jupiter.api.Assertions.*;
```

```
5
6 class SolicitudTest {
7     private Estudiante estudiante;
8
9     @BeforeEach
10    void setUp() {
11        estudiante = new Estudiante("Ana", "ana@alumnos.ucn.cl");
12    }
13
14    @Test
15    void crearSolicitudValida_dejaEstadoInicialPendiente() {
16        Solicitud solicitud = new Solicitud(estudiante, "RETIRO", "Solicitud de retiro");
17
18        assertEquals(EstadoSolicitud.PENDIENTE, solicitud.getEstado());
19    }
20
21    @Test
22    void crearSolicitudConEstudianteNulo_lanzaExcepcion() {
23        assertThrows(IllegalArgumentException.class, () ->
24            new Solicitud(null, "RETIRO", "Solicitud de retiro")
25        );
26    }
27 }
```

5.4 Estructura AAA

Arrange: preparar datos, mocks y objeto bajo prueba.
Act: ejecutar exactamente el comportamiento que interesa.
Assert: verificar salida, estado o interaccion observable.

5.5 Nombres de tests recomendados

metodoEscenario_resultadoEsperado
crearEstudianteConNombreVacio_lanzaExcepcion
cambiarEstadoValido_actualizaEstadoYPublicaEvento
calcularPromedioConTresMediciones_retornaPromedioCorrecto

5.6 Casos minimos para dominio de solicitudes

1. Crear estudiante valido.
2. Rechazar nombre vacio.
3. Rechazar correo vacio.
4. Crear solicitud valida.
5. Rechazar solicitud con estudiante nulo.
6. Rechazar tipo vacio.
7. Rechazar descripcion vacia.
8. Verificar estado inicial.
9. Cambio valido de estado.
10. Rechazar cambio si la solicitud esta finalizada.
11. Rechazar estado nulo.

5.7 Casos minimos para sensores

1. Registrar medicion cuando sensor existe.

2. Rechazar medicion si sensor no existe.
3. Calcular promedio de temperatura.
4. Calcular promedio de humedad.
5. Calcular promedio MP2.5.
6. Manejar lista vacia de mediciones.

5.8 Aserciones frecuentes

Asercion	Uso
assertEquals(a,b)	Valores esperados exactos. En JUnit 4: esperado primero.
assertEquals(a,b,delta)	Doubles: usar delta, por ejemplo 0.001.
assertTrue(cond)	Validar predicado.
assertFalse(cond)	Validar negacion.
assertNotNull(obj)	Verificar creacion.
assertThrows(...)	JUnit 5 para excepciones.
@Test(expected=...)	JUnit 4 para excepciones simples.

6 Mockito: formulario DSL

6.1 Idea central

Mockito se usa cuando una dependencia real haria el test lento, fragil o externo a la unidad: BD, red, servicio remoto, mensajeria, correo, JPA, reloj, filesystem. No mockees entidades simples si puedes crearlas normalmente.

6.2 JUnit 4 + Mockito

```

1 import org.junit.Test;
2 import org.junit.runner.RunWith;
3 import org.mockito.InjectMocks;
4 import org.mockito.Mock;
5 import org.mockito.junit.MockitoJUnitRunner;
6
7 import static org.mockito.Mockito.*;
8 import static org.junit.Assert.*;
9
10 @RunWith(MockitoJUnitRunner.class)
11 public class SolicitudServiceTest {
12     @Mock private SolicitudRepository repository;
13     @Mock private SolicitudEventPublisher publisher;
14     @Mock private NotificadorSolicitud notificador;
15
16     @InjectMocks private SolicitudService service;
17
18     @Test
19     public void cambiarEstadoValido_guardaPublicaYNotifica() {
20         Solicitud solicitud = SolicitudFixtures.pendiente();
21         when(repository.buscarPorId(10L)).thenReturn(solicitud);
22
23         service.cambiarEstado(10L, EstadoSolicitud.APROBADA);
24
25         assertEquals(EstadoSolicitud.APROBADA, solicitud.getEstado());
26         verify(repository).guardar(solicitud);
27         verify(publisher).publicarCambioEstado(solicitud);
28         verify(notificador).enviarCorreo(solicitud);
29     }
30 }
```

6.3 JUnit 5 + Mockito

```

1 import org.junit.jupiter.api.Test;
2 import org.junit.jupiter.api.extension.ExtendWith;
3 import org.mockito.InjectMocks;
4 import org.mockito.Mock;
5 import org.mockito.junit.jupiter.MockitoExtension;
6
7 import static org.mockito.Mockito.*;
8 import static org.junit.jupiter.api.Assertions.*;
9
10 @ExtendWith(MockitoExtension.class)
11 class SolicitudServiceTest {
12     @Mock SolicitudRepository repository;
13     @Mock SolicitudEventPublisher publisher;
14     @Mock NotificadorSolicitud notificador;
15
16     @InjectMocks SolicitudService service;
17
18     @Test
19     void cambiarEstadoValido_guardaPublicaYNotifica() {
20         Solicitud solicitud = SolicitudFixtures.pendiente();
21         when(repository.buscarPorId(10L)).thenReturn(solicitud);
22
23         service.cambiarEstado(10L, EstadoSolicitud.APROBADA);
24
25         assertEquals(EstadoSolicitud.APROBADA, solicitud.getEstado());
26         verify(repository).guardar(solicitud);
27         verify(publisher).publicarCambioEstado(solicitud);
28         verify(notificador).enviarCorreo(solicitud);
29     }
30 }

```

6.4 Verificar que NO se invoca dependencia externa

Caso tipico: si falla el cambio de estado, no se debe notificar ni publicar evento.

```

1 @Test
2 public void cambiarEstadoDeSolicitudFinalizada_noPublicaNiNotifica() {
3     Solicitud solicitud = SolicitudFixtures.finalizada();
4     when(repository.buscarPorId(10L)).thenReturn(solicitud);
5
6     try {
7         service.cambiarEstado(10L, EstadoSolicitud.APROBADA);
8         fail("Debia lanzar IllegalStateException");
9     } catch (IllegalStateException expected) {
10         // ok
11     }
12
13     verify(repository, never()).guardar(any());
14     verify(publisher, never()).publicarCambioEstado(any());
15     verify(notificador, never()).enviarCorreo(any());
16 }

```

6.5 Mockito para EntityManager y TypedQuery

Muy probable por la evaluacion 2025.

```

1 import jakarta.persistence.EntityManager;
2 import jakarta.persistence.TypedQuery;
3 import org.junit.Test;
4 import org.junit.runner.RunWith;
5 import org.mockito.Mock;

```



```
6 import org.mockito.junit.MockitoJUnitRunner;
7
8 import java.util.List;
9
10 import static org.junit.Assert.assertEquals;
11 import static org.mockito.Mockito.*;
12
13 @RunWith(MockitoJUnitRunner.class)
14 public class SensorRepositoryTest {
15     @Mock private EntityManager entityManager;
16     @Mock private TypedQuery<Measurement> query;
17
18     @Test
19     public void buscarMedicionesPorSensor_retornaListaMockeada() {
20         List<Measurement> mediciones = List.of(
21             new Measurement(1L, 20.0, 40.0, 10.0),
22             new Measurement(1L, 24.0, 44.0, 12.0)
23         );
24
25         when(entityManager.createQuery(anyString(), eq(Measurement.class)))
26             .thenReturn(query);
27         when(query.setParameter(eq("sensorId"), eq(1L)))
28             .thenReturn(query);
29         when(query.getResultList()).thenReturn(mediciones);
30
31         SensorRepository repository = new JpaSensorRepository(entityManager);
32         List<Measurement> resultado = repository.buscarMedicionesPorSensor(1L);
33
34         assertEquals(2, resultado.size());
35         verify(entityManager).createQuery(anyString(), eq(Measurement.class));
36         verify(query).setParameter("sensorId", 1L);
37         verify(query).getResultList();
38     }
39 }
```

6.6 ArgumentCaptor

Usa ArgumentCaptor cuando debes verificar el objeto exacto que se guardo o publico.

```
1 import org.mockito.ArgumentCaptor;
2
3 @Test
4 public void registrarMedicion_guardaMedicionConValoresCorrectos() {
5     when(sensorRepository.existsById(3L)).thenReturn(true);
6
7     service.registrarMedicion(3L, "ING", "2025-11-14T10:15:00", 23.5, 40.0, 12.3);
8
9     ArgumentCaptor<Measurement> captor = ArgumentCaptor.forClass(Measurement.class);
10    verify(measurementRepository).guardar(captor.capture());
11
12    Measurement guardada = captor.getValue();
13    assertEquals(3L, guardada.getSensorId());
14    assertEquals(23.5, guardada.getTemperature(), 0.001);
15    assertEquals(12.3, guardada.getMp25(), 0.001);
16 }
```

6.7 Errores frecuentes con Mockito

Error	Correccion
Mockear la clase bajo prueba	Mockea dependencias, no el servicio que quieres probar.
Llamar BD real	Simula repository, EntityManager o TypedQuery.
Verificar detalles irrelevantes	Verifica efectos observables: guarda, publica, notifica, retorna.
No probar caso de falla	Agrega <code>never()</code> para interacciones prohibidas.
Usar <code>any()</code> con primitivos mal	Usa <code>anyLong()</code> , <code>anyDouble()</code> o <code>eq(...)</code> .
Mezclar <code>matchers</code> y valores crudos	Si usas <code>matcher</code> en un argumento, usa <code>matchers</code> en todos: <code>eq(...)</code> .

7 Vert.x: formulario DSL

7.1 Conceptos base

Concepto	Uso
Verticle	Unidad desplegable que reacciona a eventos.
EventBus	Canal de comunicacion por direcciones.
send	Point-to-point: un consumidor recibe el mensaje.
publish	Publish/subscribe: todos los consumidores reciben copia.
request	Request/response: se espera respuesta no bloqueante.
reply	Respuesta del receptor a un request.
JsonObject	Mensaje estructurado recomendado para datos.

7.2 Arranque comun

```

1 public static void main(String[] args) {
2     Vertx vertx = Vertx.vertx();
3     vertx.deployVerticle(new MiVerticle());
4 }

```

7.3 Point-to-Point

Cuando usar

- Balanceo de carga.
- Tareas que debe procesar un unico consumidor.
- Procesamiento independiente de trabajos.

Formulario

```

1 public class Sender extends AbstractVerticle {
2     static final String ADDRESS = "tasks.process";
3
4     @Override
5     public void start() {
6         vertx.setPeriodic(1000, id -> {
7             vertx.eventBus().send(ADDRESS, "procesar tarea");
8         });
9     }
10 }

```

```

9      }
10     }
11
12     public class Receiver extends AbstractVerticle {
13         @Override
14         public void start() {
15             vertx.eventBus().consumer(Sender.ADDRESS, message -> {
16                 System.out.println("Received: " + message.body());
17             });
18         }
19     }

```

Pregunta tipica: Si hay tres consumidores en la misma direccion y se usa send, cuantos reciben el mensaje? **Respuesta:** uno solo; Vert.x elige un consumidor disponible.

7.4 Publish/Subscribe

Cuando usar

- Eventos de dominio: SolicitudCambioEstado, MedicionRegistrada.
- Notificaciones, auditoria, historial.
- Cuando el emisor no debe conocer cuantos receptores existen.

Formulario

```

1 public class SensorPublisherVerticle extends AbstractVerticle {
2     static final String ADDRESS = "air.monitor.measurements";
3
4     @Override
5     public void start() {
6         vertx.setPeriodic(1000, id -> {
7             JsonObject json = new JsonObject()
8                 .put("sensorId", 3)
9                 .put("buildingCode", "ING")
10                .put("timestamp", "2025-11-14T10:15:00")
11                .put("temperature", 23.5)
12                .put("humidity", 40.0)
13                .put("mp25", 12.3);
14
15            vertx.eventBus().publish(ADDRESS, json);
16        });
17     }
18 }
19
20 public class MeasurementConsumerVerticle extends AbstractVerticle {
21     private final MeasurementService measurementService;
22
23     public MeasurementConsumerVerticle(MeasurementService measurementService) {
24         this.measurementService = measurementService;
25     }
26
27     @Override
28     public void start() {
29         vertx.eventBus().<JsonObject>consumer(SensorPublisherVerticle.ADDRESS, msg -> {
30             JsonObject json = msg.body();
31             measurementService.registrarMedicion(
32                 json.getInteger("sensorId"),
33                 json.getString("buildingCode"),
34                 json.getString("timestamp"),
35                 json.getDouble("temperature"),
36                 json.getDouble("humidity"),

```

```

37         json.getDouble("mp25")
38     );
39 });
40 }
41 }

```

Pregunta tipica: Si hay tres consumidores en la misma direccion y se usa publish, cuantos reciben el mensaje? **Respuesta:** los tres reciben una copia.

7.5 Request/Response

Cuando usar

- Consulta interna a un servicio.
- Validacion que retorna resultado.
- RPC interno no bloqueante.

Formulario

```

1 public class RequestVerticle extends AbstractVerticle {
2     static final String ADDRESS = "solicitud.validar";
3
4     @Override
5     public void start() {
6         JsonObject request = new JsonObject().put("solicitudId", 10L);
7
8         vertx.eventBus().<JsonObject>request(ADDRESS, request, ar -> {
9             if (ar.succeeded()) {
10                 JsonObject response = ar.result().body();
11                 System.out.println("Valida: " + response.getBoolean("valida"));
12             } else {
13                 ar.cause().printStackTrace();
14             }
15         });
16     }
17 }
18
19 public class ResponseVerticle extends AbstractVerticle {
20     @Override
21     public void start() {
22         vertx.eventBus().<JsonObject>consumer(RequestVerticle.ADDRESS, msg -> {
23             Long id = msg.body().getLong("solicitudId");
24             boolean valida = id != null && id > 0;
25             msg.reply(new JsonObject().put("valida", valida));
26         });
27     }
28 }

```

7.6 Regla arquitectural para Vert.x

Un verticle es adaptador de mensajeria. Debe recibir, validar superficialmente y delegar. No debe contener reglas de negocio profundas ni abrir persistencia.

```

1 @ArchTest
2 static final ArchRule vertices_no_acceden_a_jpa =
3     noClasses().that().haveSimpleNameEndingWith("Verticle")
4         .should().dependOnClassesThat().resideInAnyPackage(
5             "jakarta.persistence..", "javax.persistence..", "..repository..")
6         .because("los verticles deben delegar en servicios, no persistir directamente")
7     ;

```

8 Plantillas de solucion por tipo de pregunta

8.1 Pregunta tipo A: “El test ArchUnit falla”

Respuesta escrita

El test falla porque una clase de una capa superior depende directamente de una capa tecnica. En particular, el controlador/verticle/web esta accediendo a infraestructura o repositorio concreto. Esto rompe la separacion de responsabilidades y aumenta acoplamiento. La solucion es introducir un servicio de aplicacion y, si corresponde, puertos/interfaces para que infraestructura implemente los detalles tecnicos.

Accion de codigo

```
Antes:
Controller -> JpaRepository
Controller -> ExternalNotifier

Despues:
Controller -> SolicitudService
SolicitudService -> SolicitudRepository (interface)
SolicitudService -> NotificadorSolicitud (interface)
JpaSolicitudRepository -> implements SolicitudRepository
EmailNotificadorSolicitud -> implements NotificadorSolicitud
```

8.2 Pregunta tipo B: “Crear test unitario”

```
1 @Test
2 public void crearEstudianteConCorreoVacio_lanzaExcepcion() {
3     try {
4         new Estudiante("Ana", "");
5         fail("Debia lanzar IllegalArgumentException");
6     } catch (IllegalArgumentException expected) {
7         assertTrue(expected.getMessage().contains("correo"));
8     }
9 }
```

8.3 Pregunta tipo C: “Crear test con Mockito”

```
1 @Test
2 public void operacionExitosa_interactuaConDependencias() {
3     Entidad entidad = Fixtures.entidadValida();
4     when(repository.buscarPorId(1L)).thenReturn(entidad);
5
6     service.ejecutar(1L);
7
8     verify(repository).guardar(entidad);
9     verify(publicador).publicar(any(Evento.class));
10    verify(notificador).notificar(entidad);
11 }
```

8.4 Pregunta tipo D: “No debe invocarse servicio externo si falla”

```
1 @Test
2 public void operacionFallida_noInvocaDependenciasExternas() {
3     when(repository.buscarPorId(1L)).thenReturn(null);
```

```

4
5    try {
6        service.ejecutar(1L);
7        fail("Debia fallar");
8    } catch (EntidadNoEncontradaException expected) {
9        // ok
10    }
11
12    verify(repository, never()).guardar(any());
13    verify(publicador, never()).publicar(any());
14    verify(notificador, never()).notificar(any());
15 }

```

8.5 Pregunta tipo E: “Migrar a Vert.x pub/sub”

1. Crear ADDRESS constante.
2. Publisher crea JsonObject con campos requeridos.
3. Publisher usa EventBus.publish(ADDRESS, json).
4. Consumer se registra con EventBus.consumer(ADDRESS, handler).
5. Consumer valida presencia de campos.
6. Consumer llama service.registrar...(...).
7. No usar EntityManager dentro del verticle.

9 Ejercicios tipo prueba con solucion orientativa

9.1 Ejercicio 1: Controller acoplado a JPA

Enunciado

La clase SolicitudController contiene:

```

1 public class SolicitudController {
2     private final JpaSolicitudRepository repository = new JpaSolicitudRepository();
3     private final EmailNotifier notifier = new EmailNotifier();
4
5     public void cambiarEstado(Long id, EstadoSolicitud estado) {
6         Solicitud solicitud = repository.findById(id);
7         solicitud.cambiarEstado(estado);
8         repository.save(solicitud);
9         notifier.send(solicitud);
10    }
11 }

```

Indique la violacion arquitectural, proponga refactor y escriba una regla ArchUnit.

Solucion

Violacion: el controlador depende directamente de infraestructura concreta. **Re-factor:** crear SolicitudService; inyectar interfaces SolicitudRepository y NotificadorSolicitud. mover la logica a service. **Regla:**

```

1 @ArchTest
2 static final ArchRule controller_no_depende_de_infra =
3     noClasses().that().resideInAPackage("..controller..")
4         .should().dependOnClassesThat().resideInAPackage("..infra..")
5         .because("el controlador no debe conocer detalles de persistencia ni servicios
6             externos");

```

9.2 Ejercicio 2: Mockito y evento de cambio de estado

Enunciado

Al cambiar correctamente el estado de una solicitud, se debe guardar, publicar evento y notificar. Si la solicitud no existe, no debe ocurrir ninguna interaccion externa.

Solucion

```
1 @RunWith(MockitoJUnitRunner.class)
2 public class SolicitudServiceTest {
3     @Mock SolicitudRepository repository;
4     @Mock SolicitudEventPublisher publisher;
5     @Mock NotificadorSolicitud notificador;
6
7     @Test
8     public void cambioValido_guardaPublicaNotifica() {
9         Solicitud solicitud = SolicitudFixtures.pendiente();
10        when(repository.buscarPorId(1L)).thenReturn(solicitud);
11        SolicitudService service = new SolicitudService(repository, publisher,
12        notificador);
13
14        service.cambiarEstado(1L, EstadoSolicitud.APROBADA);
15
16        assertEquals(EstadoSolicitud.APROBADA, solicitud.getEstado());
17        verify(repository).guardar(solicitud);
18        verify(publisher).publicarCambioEstado(solicitud);
19        verify(notificador).enviarCorreo(solicitud);
20    }
21
22    @Test
23    public void solicitudInexistente_noPublicaNiNotifica() {
24        when(repository.buscarPorId(99L)).thenReturn(null);
25        SolicitudService service = new SolicitudService(repository, publisher,
26        notificador);
27
28        try {
29            service.cambiarEstado(99L, EstadoSolicitud.APROBADA);
30            fail("Debia fallar");
31        } catch (SolicitudNoEncontradaException expected) {
32            // ok
33        }
34
35        verify(repository, never()).guardar(any());
36        verify(publisher, never()).publicarCambioEstado(any());
37        verify(notificador, never()).enviarCorreo(any());
38    }
39 }
```

9.3 Ejercicio 3: Vert.x pub/sub con sensor

Enunciado

Crear un publicador que envíe mediciones JSON y un consumidor que valide campos y delegue en MeasurementService.

Solucion

```
1 public class SensorPublisherVerticle extends AbstractVerticle {
2     static final String ADDRESS = "air.monitor.measurements";
3 }
```

```

3
4 @Override
5 public void start() {
6     JsonObject json = new JsonObject()
7         .put("sensorId", 3)
8         .put("buildingCode", "ING")
9         .put("timestamp", "2025-11-14T10:15:00")
10        .put("temperature", 23.5)
11        .put("humidity", 40.0)
12        .put("mp25", 12.3);
13
14    vertx.eventBus().publish(ADDRESS, json);
15 }
16 }
17
18 public class MeasurementConsumerVerticle extends AbstractVerticle {
19     private final MeasurementService service;
20
21     public MeasurementConsumerVerticle(MeasurementService service) {
22         this.service = service;
23     }
24
25     @Override
26     public void start() {
27         vertx.eventBus().<JsonObject>consumer(SensorPublisherVerticle.ADDRESS, msg -> {
28             JsonObject json = msg.body();
29             if (!json.containsKey("sensorId") || !json.containsKey("mp25")) {
30                 return;
31             }
32             service.registrarMedicion(
33                 json.getInteger("sensorId"),
34                 json.getString("buildingCode"),
35                 json.getString("timestamp"),
36                 json.getDouble("temperature"),
37                 json.getDouble("humidity"),
38                 json.getDouble("mp25")
39             );
40         });
41     }
42 }

```

9.4 Ejercicio 4: Poliza acoplada a XML

Enunciado

Poliza recibe XML en su constructor y usa SAX para parsear. Debe poder reutilizarse sin XML.

Solucion

```

1 public interface ConstructorPoliza {
2     Poliza construir();
3 }
4
5 public class Poliza {
6     public static Poliza construirPoliza(ConstructorPoliza constructor) {
7         return constructor.construir();
8     }
9
10    public Poliza(String nombre, String apellido, String fumador,
11        Date fechaNacimiento, String estadoCivil) {
12        // asignar atributos y validar invariantes
13    }

```



```
14 }
15
16 public class ConstructorPolizaXML implements ConstructorPoliza {
17     private final String cadenaXML;
18
19     public ConstructorPolizaXML(String cadenaXML) {
20         this.cadenaXML = cadenaXML;
21     }
22
23     @Override
24     public Poliza construir() {
25         // parsear XML aqui, no en Poliza
26         return new Poliza(nombre, apellido, fumador, fechaNacimiento, estadoCivil);
27     }
28 }
```

Justificacion: XML es un mecanismo de entrada. El dominio Poliza no debe depender de ese mecanismo.

10 Checklist final de resolucio en 90 minutos

Minuto 0-10: leer el proyecto

1. Ubica paquetes: web/controller/view, service/application, domain/model, repository/infra, vertx.
2. Ejecuta mentalmente el flujo principal.
3. Identifica imports sospechosos: EntityManager, Jpa, JavaFX, Javalin, Vertx, XML, external.

Minuto 10-25: corregir arquitectura

1. Si controller o verticle tiene logica de negocio: mover a service.
2. Si service depende de implementacion tecnica: introducir interfaz/puerto.
3. Si domain depende de framework: mover anotaciones/logica tecnica a infra o aceptar solo si el enunciado permite model JPA.

Minuto 25-45: ArchUnit

1. Escribe una regla que refleje el problema principal.
2. Escribe una regla adicional sencilla y defendible.
3. Revisa @AnalyzeClasses y patrones de paquetes.

Minuto 45-65: JUnit y Mockito

1. Tests de dominio: constructores, validaciones, estados.
2. Tests de service: caso exitoso y caso de falla.
3. Mockito: verify de guardado/publicacion/notificacion; never en caso fallido.

Minuto 65-80: Vert.x

1. Decide el patron: publish para eventos, send para una tarea, request si esperas respuesta.
2. Usa JsonObject para mensajes estructurados.
3. Consumer delega a service.

Minuto 80-90: defensa y limpieza

1. No dejar imports tecnicos en capas prohibidas.
2. No dejar tests que dependan de SQLite real si son unitarios.
3. Escribir justificacion breve: "se redujo acoplamiento porque...".

11 Banco rapido de respuestas teoricas

¿Por que ArchUnit?

Porque una arquitectura documentada no se conserva sola. ArchUnit convierte decisiones arquitecturales en tests ejecutables, por lo que una dependencia indebida puede fallar en `mvn test` antes de llegar a merge o entrega.

¿Por que usar mocks?

Porque un test unitario debe aislar la unidad. Si se llama una BD real, red, servicio externo o broker real, ya no se esta probando solo la unidad: se mezcla con integracion.

¿Por que publish/subscribe para cambio de estado?

Porque "cambio de estado" es un evento. El emisor no necesita saber si escuchan historial, auditoria, notificacion o estadisticas. Esto reduce acoplamiento y permite agregar consumidores sin modificar el caso de uso principal.

¿Por que request/response dentro de EventBus?

Porque permite una consulta o validacion tipo RPC interna sin bloquear el hilo. Se usa cuando el emisor necesita una respuesta del receptor.

¿Por que point-to-point?

Porque una tarea debe ser procesada por un unico consumidor. Si existen varios consumers, se puede repartir carga.

¿Por que no microservicios de inmediato?

Porque agregan complejidad distribuida: latencia, despliegue de multiples procesos, observabilidad y transacciones distribuidas. Conviene comenzar monolitico y mirar si rendimiento o releases lo justifican.

12 Resumen ultra-condensado

ArchUnit

```
1 @AnalyzeClasses(packages = "cl.ucn.app")
2 class ArchitectureTest {
3     @ArchTest
4     static final ArchRule regla =
5         noClasses().that().resideInAPackage("..
6             controller..")
7             .should().dependOnClassesThat().
8             resideInAPackage("..infra..");
9 }
```

JUnit 4

```
1 @Test
2 public void caso_resultado() {
3     assertEquals(esperado, obtenido);
4 }
5
6 @Test(expected = IllegalArgumentException.
7     class)
8 public void invalido_lanza() { ... }
```

Mockito

```
1 when(repo.findById(1L)).thenReturn(obj);
2 service.ejecutar(1L);
3 verify(repo).save(obj);
4 verify(notifier, never()).send(any());
```

```
1 eventBus.publish("events", json);
2 eventBus.<JsonObject>consumer("events", h
  -> service.exec(h.body()));
```

Vert.x send

```
1 eventBus.send("addr", msg);
2 eventBus.consumer("addr", h -> { ... });
```

Vert.x request

```
1 eventBus.<String>request("addr", req, ar
  -> {
2   if (ar.succeeded()) System.out.println(
    ar.result().body());
3 });
4
5 consumer.handler(msg -> msg.reply("ok"));
```

Vert.x publish

13 Fuentes revisadas

- Repositorio formativo 2026: [FormativoII-PSP-2026](#).
- Evaluacion II 2025: [Evaluacion-II-II-2025](#).
- Evaluacion II 2024: [EvaluacionII-PSP-II-2024](#).
- Formativa II 2025: [Formativa-II-II-2025](#).
- PDF de clases: Arquitectura de Software, Testing de Sistemas, Programacion Reactiva y ArchUnit.